

Assignment 6: GUI-Based Multi-Client Chat Application Using TCP

Name : Piyush Kumar

Roll : CSB24051

1. Objective

The objective of this assignment is to convert the terminal-based TCP chat application developed in Assignment 5 into a Graphical User Interface (GUI) based desktop application using Python Tkinter. The existing TCP socket communication and multithreaded server implementation are reused with minimal modifications while replacing the terminal client with a user-friendly graphical client.

2. GUI Design

The graphical user interface was developed using Python's Tkinter library. The GUI consists of two major windows:

a. Login Window

The login window is displayed when the application starts.

It contains:

- Username Entry Box
 - Server IP Entry Box
-

-
- Connect Button

The user enters the desired username and the IP address of the chat server. The client sends a login request to the server using the TCP protocol. If the username is unique, the server accepts the connection and the main chat window opens. Otherwise, an error message is displayed.

b. Main Chat Window

After successful authentication, the main chat interface is displayed.

The window consists of:

- Scrollable chat display area
- Message input field
- Send button
- Disconnect button
- Online users panel
- Connection status indicator

3. System Architecture

The application follows a Client-Server Architecture.

The server remains active continuously and listens on TCP port 5000. Every client establishes an independent TCP connection with the server. Whenever a client sends a message, the server receives it and forwards it to all connected clients or to the intended recipient in case of private messaging.

Each client runs two separate activities simultaneously:

- GUI Event Loop
- Background Receiving Thread

The background thread continuously listens for incoming messages from the server while the GUI thread handles user interactions. This separation prevents the graphical interface from freezing during communication.

4. Components Reused from Assignment 5

Most of the networking implementation developed in Assignment 5 was reused without significant modification. Only the terminal-based client interface was replaced with a graphical interface.

The following components were reused:

Component	Reused
TCP Socket Programming	Yes
Server Implementation	Yes
Broadcast Messaging	Yes
Private Messaging	Yes
Chat History Logging	Yes
Client Connection Handling	Yes
CSV Logging	Yes

5. GUI Design Decisions

The Graphical User Interface (GUI) was developed using Python's Tkinter library to provide an easy-to-use and interactive chat application. The primary objective was to replace the terminal-based client from Assignment 5 while keeping the networking logic unchanged.

The GUI was designed with simplicity, responsiveness, and usability in mind. A login window is displayed initially to collect the username and server IP address. After successful authentication, the main chat window opens, providing all communication features in a single interface.

GUI Features

- Login Window
- Scrollable Chat Area
- Online User List
- Broadcast Messaging
- Private Messaging
- Automatic Message Scrolling
- Background Message Receiving
- Connection Status Indicator
- Disconnect Button

6. Testing Results

The application was tested using Mininet with one server and four clients.

1. user login



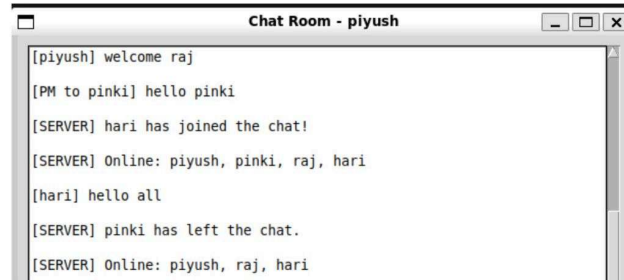
2. Broadcast Messaging

Observation

Piyush : Hello Everyone

appears on every connected client.

Result : **PASS**



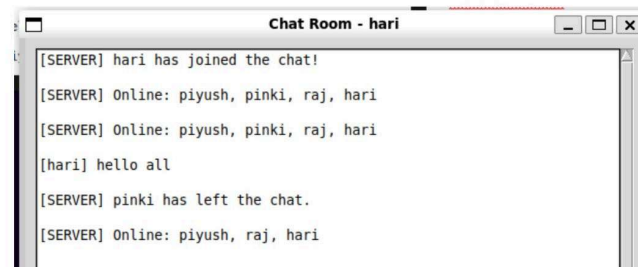
Chat Room - piyush

```
[piyush] welcome raj
[PM to pinki] hello pinki
[SERVER] hari has joined the chat!
[SERVER] Online: piyush, pinki, raj, hari
[hari] hello all
[SERVER] pinki has left the chat.
[SERVER] Online: piyush, raj, hari
```



Chat Room - pinki

```
[pinkie] nello
[SERVER] raj has joined the chat!
[SERVER] Online: piyush, pinki, raj
[raj] hello dear
[piyush] welcome raj
[PM from piyush] hello pinki
```



Chat Room - hari

```
[SERVER] hari has joined the chat!
[SERVER] Online: piyush, pinki, raj, hari
[SERVER] Online: piyush, pinki, raj, hari
[hari] hello all
[SERVER] pinki has left the chat.
[SERVER] Online: piyush, raj, hari
```

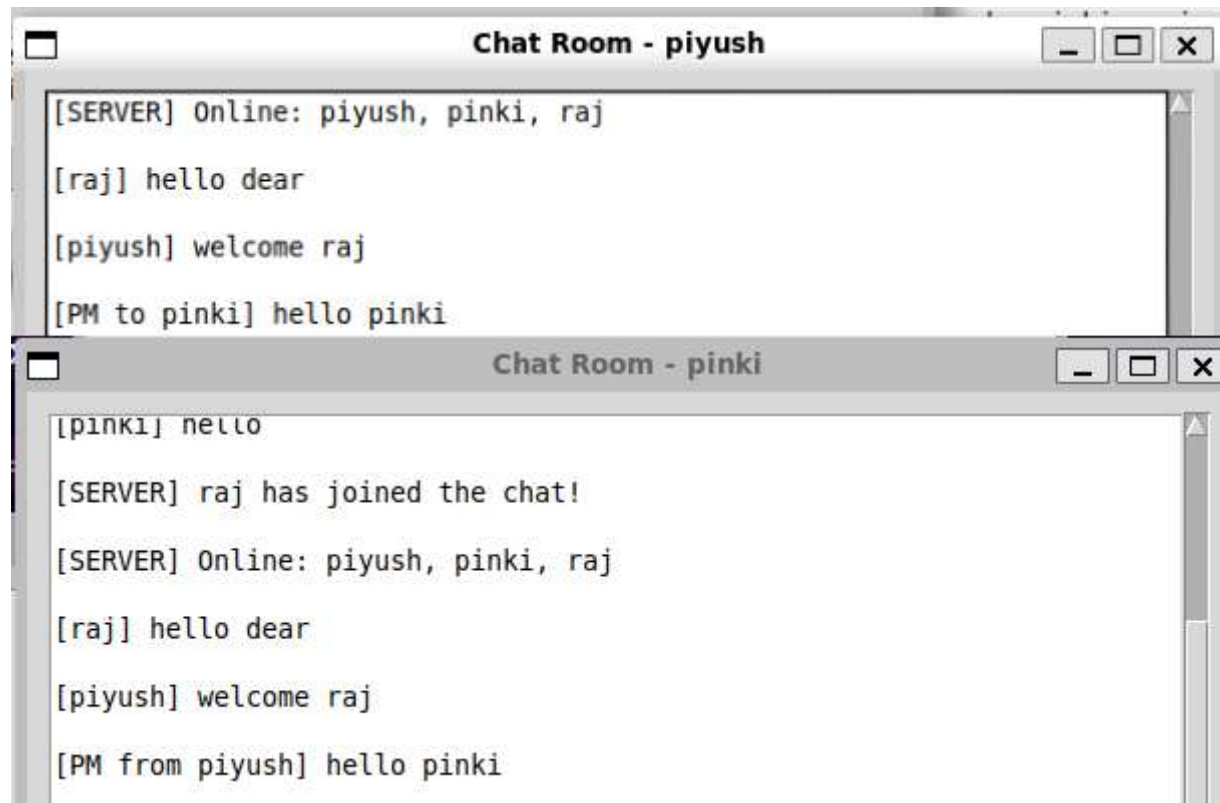


Chat Room - raj

```
[raj] hello dear
[piyush] welcome raj
[SERVER] hari has joined the chat!
[SERVER] Online: piyush, pinki, raj, hari
[hari] hello all
[SERVER] pinki has left the chat.
```

3. Private messaging

Only the selected user receives the message.



4. User joining

Verify join notifications.



5. User leaving

Verify leave notifications.



6. Online User List

```
[SERVER] piyush has joined the chat!
[SERVER] Your last messages:
  (14:42:57) [broadcast -> ALL] pm <piyush_2> hello
  (14:55:38) [private -> piyush_2] hello, this is pm from h2 to u h3
  (11:37:30) [broadcast -> ALL] hello
  (11:38:37) [broadcast -> ALL] welcome raj
  (11:42:17) [private -> pinki] hello pinki
[piyush] /list/list
[SERVER] pinki has joined the chat!
[SERVER] Online: piyush, pinki
[SERVER] hari has joined the chat!
[SERVER] Online: piyush, pinki, hari
[SERVER] raj has joined the chat!
[SERVER] Online: piyush, pinki, hari, raj
[SERVER] hari has left the chat.
[SERVER] Online: piyush, pinki, raj
```

7. Wireshark Verification

using filter tcp.port == 5000.

1. client connection

The TCP three-way handshake (SYN, SYN-ACK, ACK) was observed before data transmission.

Login Request:

The client transmitted the username to the server over the established TCP connection.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.2	10.0.0.1	TCP	72	57446 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=3641476097 TSecr=0 WS=512
2	0.000129173	10.0.0.1	10.0.0.2	TCP	74	5000 → 36446 [SYN, ACK] Seq=0 Ack=1 Win=42440 Len=0 MSS=1460 SACK_PERM TSval=4231182694 TSecr=3641476097 WS=512
3	0.000365360	10.0.0.2	10.0.0.1	TCP	66	36446 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=3641476098 TSecr=4231182694

2. broadcast message

The server forwarded the broadcast message to every connected client.

```
message_17_from_1
[Piyush_2]: message_14_from_2
[Piyush_0]: message_18_from_0
[OK]
[Piyush_0]: message_19_from_0
```

3. private message

The private message was transmitted only to the intended receiver.

```
/msg perf_user_2 hello_15
[Piyush_2]: message_12_from_2
[Piyush_0]: message_16_from_0
```

4. client disconnection

A FIN packet followed by ACK packets was observed during graceful client termination

```
message_49_from_1  
*** Piyush_0 has left the chat ***  
[OK]  
[OK]  
[Piyush_2]: message_47_from_2  
/quit
```

8. Reflection Answers

Q1. Why should networking logic and GUI code be separated?

Networking logic and GUI code should be separated to improve modularity, maintainability, and scalability of the application. The networking module is responsible for handling TCP socket communication, message transmission, client management, and server interaction, whereas the GUI module is responsible for displaying information and receiving user input. By separating these two components, changes made to the graphical interface do not affect the networking functionality and vice versa. This modular approach simplifies debugging, testing, and future enhancements.

Q2. Why is a background thread required?

Socket operations such as `recv()` are blocking operations. If these operations are executed in the main GUI thread, the graphical interface becomes unresponsive while waiting for incoming messages.

To solve this problem, a separate background thread continuously listens for messages from the server. The main GUI thread remains free to process user interactions such as

typing messages, clicking buttons, and updating widgets. This ensures a responsive and smooth user experience.

Q3. Which components from Assignment 5 were reused?

The following components developed in Assignment 5 were reused without significant modifications:

- TCP socket communication
- Client connection handling
- Broadcast messaging
- Private messaging
- Chat history logging
- CSV logging
- Online user management

Only the terminal-based client interface was replaced by a graphical interface developed using Tkinter.

Q4. How did the GUI improve usability?

The graphical interface significantly improved usability compared to the terminal-based application.

Improvements include:

- Easy login through graphical input fields.
- Buttons instead of typed commands.
- Scrollable message display.
- Automatic update of online users.
- Visual connection status.

-
- Simple private messaging using user selection.
 - Better readability of chat history.
 - User-friendly interaction suitable for non-technical users.

Overall, the GUI provides a more intuitive and efficient communication experience.

Q5. Suggest one future enhancement.

One possible enhancement is the implementation of file sharing and multimedia messaging. Users could send images, documents, audio, and video files through the chat application. Additional features such as message encryption, emoji support, user authentication with passwords, and group chat functionality could further improve security and usability.

9. Conclusion

In this assignment, a GUI-based multi-client chat application was successfully developed using Python Tkinter and TCP sockets. The terminal-based client from Assignment 5 was replaced with a graphical desktop interface while reusing the existing server implementation.

The application supports multiple simultaneous client connections using multithreading. Users can communicate through broadcast and private messaging, view online users, receive join and leave notifications, and disconnect gracefully. A background thread was implemented to receive incoming messages without affecting GUI responsiveness.